



第七章

动态规划

—A powerful method for combinatorial problems

华中农业大学信息学院

主要内容



- 基本概念
 - 简介
 - 一个例子
 - 动态规划范式
- 用动态规划策略处理的问题
 - 背包问题
 - 最长公共子序列问题
 - 矩阵链相乘

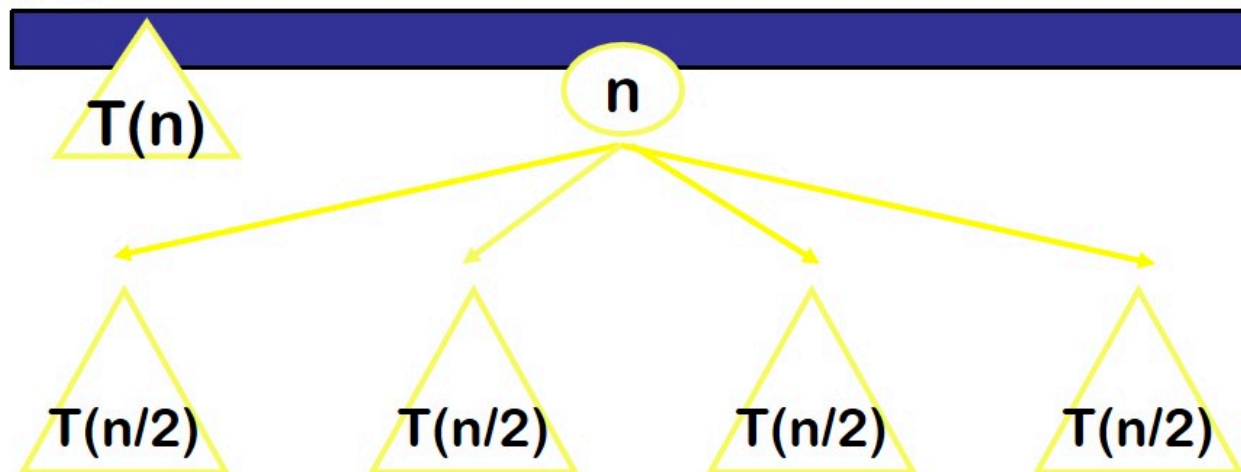
2025-10-15

²
华中农业大学信息学院



7.1 简介

- 动态规划
- 动态规划算法与分治法类似，其基本思想也是将待求解问题分解成若干个子问题。



2025-10-15

3
华中农业大学信息学院



7.1 简介

- 但是经分解得到的子问题往往不是互相独立的。不同子问题的数目常常只有多项式量级。在用分治法求解时，有些子问题被重复计算了许多次。
- 如果能够保存已解决的子问题的答案，而在需要时再找出已求得的答案，就可以避免大量重复计算，从而得到多项式时间算法。

Those who cannot remember the past are
doomed to repeat it.

-----George Santayana,

(1905)

2025-10-15

4
华中农业大学信息学院



例子（求Fibonacci 序列）

- 定义Fibonacci 序列 f_n , 序列中的每一个数是它前面两个数的和:

$$f_1 = 1, f_2 = 1, f_3 = 2, f_4 = 3, f_5 = 5, f_6 = 8, f_7 = 13, \dots$$

- 我们来看这个序列的递归定义

$$f(n) = \begin{cases} 1 & \text{if } n=1 \text{ or } n=2 \\ f(n-1) + f(n-2) & \text{if } n \geq 3 \end{cases}$$

- 这个定义暗示一个看起来像下面那样表示的过程:

- :
 1. procedure $f(n)$
 2. if $(n=1)$ or $(n=2)$ then return 1
 3. else return $f(n-1)+f(n-2)$

? 传统的解决方法是什么?



递归算法的缺陷

- 我们把递推式做适当的展开:

$$\begin{aligned}f(n) &= f(n-1) + f(n-2) \\&= 2f(n-2) + f(n-3) \\&= 3f(n-3) + 2f(n-4)\end{aligned}$$

- 这导致了巨大数量的重复调用。假设计算 $f(1)$ 和 $f(2)$ 需要一个单位时间,那么这个过程的时间复杂性可以表示为

$$T(n) = \begin{cases} 1 & \text{if } n=1 \text{ or } n=2 \\ T(n-1) + T(n-2) & \text{if } n \geq 3 \end{cases}$$

- 递推式的解是: $T(n) = f(n) \approx \Theta(\phi^n)$ $\phi = (1 + \sqrt{5})/2 \approx 1.618\ 03$
- 可以看出, 计算 $f(n)$ 所需要的运行时间对于 n 的值是**指数**的,

? Much more efficient solution?



一种有效算法

- 采取**自底向上的方式**递推求值，并把**中间结果**存储起来以便以后用来计算所要求的解。利用这种技术可以设计出特别有效的算法来解决许多组合最优化问题。
- 对于Fibonacci 函数来说，如果从 f_1 开始自底向上地计算直至到 f_n ，只需要 $\Theta(n)$ 时间和 $\Theta(1)$ 空间。和上面的方法相比，可以很大程度地降低时间复杂度。

2025-10-15

7
华中农业大学信息学院



动态规划和分治方法

- 共同点:
 - 动态规划与分治方法相似，通过合并多个子问题的解来解决整体问题。

2025-10-15

8
华中农业大学信息学院

动态规划和分治方法



- 区别:

1. 分治法是把大问题分解成一些相互独立的子问题，递归的求解这些子问题然后将他们合并来得到整个问题的解
2. 动态规划是通过组合子问题的解来解决整个大问题。各个子问题不是独立的，也就是各个子问题包含公共子问题。它可以避免遇到的子问题的重复求解。

2025-10-15

9
华中农业大学信息学院



最优化问题

- 通常采用动态规划对问题进行**优化**。对于一个问题，可以有很多可能的解决方案。每个解决方案有一个值，我们希望找到一个最佳的（最小或最大）值对应的解决方案，我们称这样的解决办法为**最优的解决方案**。

2025-10-15

10
华中农业大学信息学院



动态规划基本步骤

- 找出最优解的性质，并刻画其结构特征。
- 递归地定义最优值。
- 以自底向上的方式计算出最优值。
- 根据计算最优值时得到的信息，构造最优解。

2025-10-15

11
华中农业大学信息学院



动态规划算法的基本要素1(最优子结构)

- 矩阵连乘计算次序问题的最优解包含着其子问题的最优解。这种性质称为**最优子结构性质**。
- 在分析问题的最优子结构性质时，所用的方法具有普遍性：首先假设由问题的最优解导出的子问题的解不是最优的，然后再设法说明在这个假设下可构造出比原问题最优解更好的解，从而导致矛盾。
- 利用问题的最优子结构性质，以自底向上的方式递归地从子问题的最优解逐步构造出整个问题的最优解。
最优子结构是问题能用动态规划算法求解的前提。

同一个问题可以有多种方式刻画它的最优子结构，有些表示方法的求解速度更快（空间占用小，问题的维度低）



动态规划算法的基本要素2(重叠子问题)

- 递归算法求解问题时，每次产生的子问题并不总是新问题，有些子问题被反复计算多次。这种性质称为**子问题的重叠性质**。
- 动态规划算法，对每一个子问题只解一次，而后将其解保存在一个表格中，当再次需要解此子问题时，只是简单地用常数时间查看一下结果。
- 通常不同的子问题个数随问题的大小呈多项式增长。因此用动态规划算法只需要多项式时间，从而获得较高的解题效率。

2025-10-15

13
华中农业大学信息学院



7.1 背包问题

问题描述:

- 背包问题可以定义如下
 - 设 $U = \{u_1, u_2, \dots, u_n\}$ 是一个准备放入容量为 C 的背包中的 n 项物品的集合。对于 $1 \leq j \leq n$ ，令 s_j 和 v_j 分别为第 j 项物品的体积和价值，这里， C ， s_j ， v_j 和 j 都是正整数
 - 我们要解决的问题是用 U 中的一些物品来装满背包，这些物品的总体积不超过 C ，然而要使它们的总价值最大。

2025-10-15

14
华中农业大学信息学院



背包问题

- 更形式地, 给出有 n 项物品的 U , 我们要找出一个子集合 $\subseteq S$ U 使得 $\sum_{u_i \in S} v_i$

在约束条件 $\sum_{u_i \in S} s_i \leq C$ 下最大。



基本思想

- 设 $V[i,j]$ 用来表示从前 i 项 $\{u_1, \dots, u_i\}$ 中取出来的装入体积为 j 的背包的物品的最大价值。这里， i 的范围是从0到 n ， j 的范围是0到 C 。
- $V[0,j]$ 对于所有 j 的值是0，这是由于背包中什么也没有。另一方面， $V[i,0]$ 对于所有 i 的值为0，因为没有东西可放到为0的背包里。



基本思想

- 背包问题特点是：每种物品仅有一件，可以选择放或不放。
- 用子问题定义状态：即 $V[i,j]$ 表示前 i 件物品恰放入一个容量为 j 的背包可以获得的**最大价值**。则其状态转移方程便是： $V[i,j]=\max\{V[i-1,j], V[i-1, j-s_i]+v_i\}$ 。
- 针对“将前 i 件物品放入容量为 v 的背包中”这个子问题，若只考虑第 i 件物品的策略（放或不放），那么就可以转化为一个只牵扯前 $i-1$ 件物品的问题。
 - ✓ 如果不放第 i 件物品(因为放不下)，那么问题就转化为“前 $i-1$ 件物品放入容量为 j 的背包中”，价值为 $V[i-1,j]$ ；
 - ✓ 如果放第 i 件物品，那么问题就转化为“前 $i-1$ 件物品放入剩下的容量为 $j-s_i$ 的背包中”，此时能获得的最大价值就是 $V[i-1, j-s_i]$ 再加上通过放入第 i 件物品获得的价值 v_i 。

2025-10-15

17
华中农业大学信息学院



观察结论 7.2

$V[i,j]$ 是下面两个量的最大值:

1. $V[i-1,j]$:仅用最优的方法取自 $\{u_1, u_2, \dots, u_{i-1}\}$ 的物品去装入体积为 j 的背包所得到的价值最大值。
2. $V[i-1,j-s_i]+v_i$:用最优的方法取自 $\{u_1, u_2, \dots, u_{i-1}\}$ 的物品去装入体积为 $j-s_i$ 的背包所得到的价值最大值加上物品 u_i 的价值。这仅应用于如果 $j \geq s_i$ 以及它等于把物品 u_i 加到背包上的情况。

$$V[i,j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ V[i-1,j] & \text{if } j < s_i \\ \max\{V[i-1,j], V[i-1,j-s_i]+v_i\} & \text{if } i > 0 \text{ and } j \geq s_i \end{cases}$$

2025-10-15

18
华中农业大学信息学院



算法7.4 KNAPSACK

输入: 物品集合 $U=\{u_1, u_2, \dots, u_n\}$, 体积分别为 $s_1, s_2, \dots, s_n$, 价值分别为 $v_1, v_2, \dots, v_n$, 容量为 C 的背包。

输出: $\sum_{u_i \in S} v_i$ 的最大总价值, 且满足 $\sum_{u_i \in S} s_i \leq C$, 其中 $S \subseteq U$.

```
1. for  $i \leftarrow 0$  to  $n$ 
2.      $V[i, 0] \leftarrow 0$ 
3. end for
4. for  $j \leftarrow 0$  to  $C$ 
5.      $V[0, j] \leftarrow 0$ 
6. end for
7. for  $i \leftarrow 1$  to  $n$ 
8.     for  $j \leftarrow 1$  to  $C$ 
9.          $V[i, j] \leftarrow V[i-1, j]$ 
10.        if  $s_i \leq j$  then  $V[i, j] \leftarrow \max\{V[i, j], V[i-1, j-s_i] + v_i\}$ 
11.    end for
12. end for
13. return  $V[n, C]$ 
```

2025-10-15

19
华中农业大学信息学院



例子

- 容量: 9; 4 种体积的物品: 2, 3, 4, 5; 价值分别为: 3, 4, 5, 7.

The maximum value is 12. Consider put 2 in the first knapsack and 3, 4 in the second knapsack.

	0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	0	0	0	0	0	0
1	0	0	3	3	3	3	3	3	3	3
2	0	0	3	4	4	7	7	7	7	7
3	0	0	3	4	5	7	8	9	9	12
4	0	0	3	4	5	7	8	10	11	12

2025-10-15

20
华中农业大学信息学院



算法分析

- 很明显，由于计算表的每一项需要 $\Theta(1)$ 时间，算法的时间复杂度恰好是表的大小 $\Theta(nC)$ 。
- 由于计算当前行只需要上一次计算的行，因此对算法稍作修改可以使其只需要 $\Theta(C)$ 的空间。
- 定理7.3
 - 背包问题的最优解能够在 $\Theta(nC)$ 的时间和 $\Theta(C)$ 的空间内得到。



7.2 最长公共子序列

- 若给定序列 $X=\{x_1, x_2, \dots, x_m\}$ ，则另一序列 $Z=\{z_1, z_2, \dots, z_k\}$ ，是X的**子序列**是指存在一个严格递增下标序列 $\{i_1, i_2, \dots, i_k\}$ 使得对于所有 $j=1, 2, \dots, k$ 有： $z_j=x_{i_j}$ 。例如，序列 $Z=\{B, C, D, B\}$ 是序列 $X=\{A, B, C, B, D, A, B\}$ 的子序列，相应的递增下标序列为 $\{2, 3, 5, 7\}$ 。
- 给定2个序列X和Y，当另一序列Z既是X的子序列又是Y的子序列时，称Z是序列X和Y的**公共子序列**。
- 给定2个序列 $X=\{x_1, x_2, \dots, x_m\}$ 和 $Y=\{y_1, y_2, \dots, y_n\}$ ，找出X和Y的**最长公共子序列**。



最长公共子序列长度

- 在字母表 E 上，分别给出两个长度为 n 和 m 的字符串 A 和 B ，确定在 A 和 B 中最长公共子序列的长度。
- 定义 $A = a_1, a_2, \dots, a_n$ 的子序列是一个形式为 $a_{i_1}, a_{i_2}, \dots, a_{i_k}$ 的字符串，其中每个 i_j 都在1和 n 之间，并且 $1 \leq i_1 < i_2 < \dots < i_k \leq n$ 。
- 如果 $\Sigma = \{x, y, z\}$ ， $A = zxyxyz$ 和 $B = xy yzx$ ，那么 xyy 同时是 A 和 B 的长度为3的子序列。
- 然而，它不是 A 和 B 最长的公共子序列，因为字符串 $xyyz$ ，也是 A 和 B 公共的子序列，由于这两个字符串没有比4更长的公共子序列，因此， A 和 B 的最长的公共子序列的长度是4。

LCS 问题



问题描述

- 在最长公共子序列问题中 (**longest-common-subsequence (LCS) problem**) , 给定两个序列 $A = a_1, a_2, \dots, a_n$ and $B = b_1, b_2, \dots, b_m$, 并希望找到A和B中的最长公共子序列以及序列长度。.



蛮力搜索算法

- 列举A所有的 2^n 个子序列
- 对于每一个子序列在 $\Theta(m)$ 时间内来确定它是否也是B的子序列
- 很明显，此算法的时间复杂性是 $\Theta(m2^n)$,是指数复杂性的。

2025-10-15

25
华中农业大学信息学院



动态规划技术

- 设序列 $A = \{a_1, a_2, \dots, a_n\}$ 和 $B = \{b_1, b_2, \dots, b_m\}$ 的最长公共子序列为 $C = \{c_1, c_2, \dots, c_k\}$ ，则
 - (1) 若 $a_n = b_m$ ，则 $c_k = a_n = b_m$ ，且 C_{k-1} 是 A_{n-1} 和 B_{m-1} 的最长公共子序列。
 - (2) 若 $a_n \neq b_m$ 且 $c_k \neq a_n$ ，则 C 是 A_{n-1} 和 B 的最长公共子序列。
 - (3) 若 $a_n \neq b_m$ 且 $c_k \neq b_m$ ，则 C 是 A 和 B_{m-1} 的最长公共子序列。
- 由此可见，**2**个序列的最长公共子序列包含了这**2**个序列的前缀的最长公共子序列。因此，最长公共子序列问题具有最优子结构性质。



动态规划技术

- 设序列 $A=\{a_1, a_2, \dots, a_n\}$ 和 $B=\{b_1, b_2, \dots, b_m\}$, 令 $L[i, j]$ 表示 $a_1, a_2, \dots, a_i$ 和 $b_1, b_2, \dots, b_j$ 的最长公共子序列的**长度**。
- 观察结论 7.1
如果 i 和 j 都大于0, 那么
 - 若 $a_i = b_j, L[i, j] = L[i-1, j-1] + 1$;
 - 若 $a_i \neq b_j, L[i, j] = \max\{L[i, j-1], L[i-1, j]\}$ 。



动态规划技术

- 下面计算A和B的最长公共子序列长度的递推式。
可以从观察结论7.1立即得出：

$$c[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0, \\ c[i-1, j-1] + 1 & \text{if } i, j > 0 \text{ and } x_i = y_j, \\ \max(c[i, j-1], c[i-1, j]) & \text{if } i, j > 0 \text{ and } x_i \neq y_j. \end{cases}$$



算法7.1 LCS

输入:字母表 Σ 上的两个字符串A和B, 长度分别为n和m

输出: A和B最长公共子序列的长度。

1. **for** $i \leftarrow 0$ **to** n // $j=0$ 时, $L[i,j]=0$
2. $L[i,0] \leftarrow 0$
3. **end for**
4. **for** $j \leftarrow 0$ **to** m // $i=0$ 时, $L[i,j]=0$
5. $L[0,j] \leftarrow 0$
6. **end for**
7. **for** $i \leftarrow 0$ **to** n
8. **for** $j \leftarrow 0$ **to** m
9. **if** $a_i = b_j$ **then** $L[i,j] \leftarrow L[i-1,j-1] + 1$
10. **else** $L[i,j] \leftarrow \max\{L[i,j-1], L[i-1,j]\}$
11. **end if**
12. **end for**
13. **end for**
14. **return** $L[n,m]$

华中农业大学信息学院



- Example: $X = A, B, C, B, D, A, B$ and $Y = B, D, C, A, B, A$.

		j	0	1	2	3	4	5	6
i	x_i	y_j		B	D	C	A	B	A
0	x_i		0	0	0	0	0	0	0
1	A		0	↑	↑	↑	↖1	←1	↖1
2	B		0	↖1	←1	←1	↑1	↖2	←2
3	C		0	↑1	↑1	↖2	←2	↑2	↑2
4	B		0	↖1	↑1	↑2	↑2	↖3	←3
5	D		0	↑1	↖2	↑2	↑2	↑3	↑3
6	A		0	↑1	↑2	↑2	↖3	↑3	↖4
7	B		0	↖1	↑2	↑2	↑3	↖4	↑4

2025-10-15

30
华中农业大学信息学院



算法分析

- 定理7.1
 - 最长公共子序列问题的最优解能够在 $\Theta(nm)$ 时间和 $\Theta(\min\{m,n\})$ 空间内得到。



7.3 矩阵链相乘

? 问题描述:

- 给定 n 个矩阵 $\{A_1, A_2, \dots, A_n\}$, 其中 A_i 与 A_{i+1} 是可乘的, $i=1, 2, \dots, n-1$ 。要算出这 n 个矩阵的连乘积 $A_1 A_2 \dots A_n$ 。
- 由于矩阵乘法满足结合律, 故计算矩阵的连乘积可以有許多不同的计算次序。这种计算次序可以用加括号的方式来确定。若一个矩阵连乘积的计算次序完全确定, 也就是说该连乘积已完全加括号, 则可以依此次序反复调用2个矩阵相乘的标准算法计算出矩阵连乘积。
- 例如, 矩阵连乘积 $A_1 A_2 A_3 A_4$ 有5种不同的完全加括号的方式
 $(A_1 (A_2 (A_3 A_4)))$,
 $(A_1 ((A_2 A_3) A_4))$,
 $((A_1 A_2) (A_3 A_4))$,
 $((A_1 (A_2 A_3)) A_4)$,
 $((A_1 A_2) A_3) A_4$.

2025-10-15

32
华中农业大学信息学院



矩阵链相乘

- 每一种完全加括号的方式对应于一个矩阵连乘积的计算次序，这决定着作乘积所需要的计算量。
- 首先考虑两个矩阵相乘的成本。标准的算法由下面的伪代码给出：

```
Algorithm MATRIX-MULTIPLY(A, B)

if  $columns[A] \neq rows[B]$ 
then error "incompatible dimensions"
else for  $i \leftarrow 1$  to  $rows[A]$ 
    for  $j \leftarrow 1$  to  $columns[B]$ 
         $C[i, j] \leftarrow 0$ 
        for  $k \leftarrow 1$  to  $columns[A]$ 
             $C[i, j] \leftarrow C[i, j] + A[i, k] \cdot B[k, j]$ 
        end for
    end for
end for
return C
```

相乘的两个矩阵必须是兼容的 (*compatible*) :即A矩阵的列数必须等于B矩阵的行. 若A是一个 $p \times q$ 矩阵, B是一个 $q \times r$ 矩阵, 则计算其乘积 $C=AB$ 的标准算法中, 需要进行 pqr 次数乘。



矩阵链相乘

- 先考察3个矩阵 $\{A_1, A_2, A_3\}$ 连乘的情况。设这三个矩阵的维数分别为 10×100 , 100×5 , and 5×50 。

加括号方式	第一次相乘次数	第二次相乘次数	总相乘次数
$((A_1 A_2) A_3)$	$10 \cdot 100 \cdot 5 = 5000$	$10 \cdot 5 \cdot 50 = 2500$	7500
$(A_1 (A_2 A_3))$	$100 \cdot 5 \cdot 50 = 25000$	$10 \cdot 100 \cdot 50 = 50000$	75000



问题

如何确定计算矩阵连乘积 $A_1 A_2 \dots A_n$ 的计算次序（完全加括号方式），使得依此次序计算矩阵连乘积需要的数乘次数最少。

2025-10-15

34
华中农业大学信息学院



蛮力方法: 穷举所有的计算次序

基本思路

- 对于 n 个矩阵的连乘积, 设其不同的计算次序为 $P(n)$
- 当 $n=1$, 只有一个矩阵, 因此 $P(n)=1$
- 当 $n \geq 2$, 对于前 k 个矩阵有 $P(k)$ 种方法放置括号。对于 $P(k)$ 中的每一种方法, 可对余下的 $P(n-k)$ 个矩阵放置括号, 总共有 $P(k)P(n-k)$ 种方法。由于可以假设 k 是1到 $n-1$ 中的任意值, 对于 n 个矩阵放置括号的所有方法数由下面的和式给出:

$$P(n) = \begin{cases} 1 & \text{if } n=1. \\ \sum_{k=1}^{n-1} P(k)P(n-k) & \text{if } n \geq 2. \end{cases}$$

2025-10-15

35
华中农业大学信息学院



蛮力方法: 穷举所有的计算次序

- 递推式的解 p133

$$P(n) = \frac{1}{n} \binom{2n-2}{n-1} = \frac{(2n-2)!}{n((n-1)!)^2} \approx \frac{4^n}{4\sqrt{\pi}n^{1.5}}$$

- 由于对于每个括弧号表达式, 找到数量乘法次数的时间耗费是 $\Theta(n)$, 这样用蛮力方法可以求得找到 n 个矩阵相乘的最优方法所需的运行时间是 $\Omega(4^n/n^{0.5})$, 它甚至对于一个中等规模的 n 值也是不切实际的。



最优子结构

- 将这个计算次序在矩阵 $A(k)$ 和 $A(k+1)$ 之间将矩阵链断开，则生成两个矩阵链 $A_1 \dots A_k$ 和 $A_{k+1} \dots A_n$ 。计算 $A_1 \dots A_n$ 的最优次序所包含的计算矩阵子链 $A_1 \dots A_k$ 和 $A_{k+1} \dots A_n$ 的次序也是最优的。
- 矩阵子链的所耗费的计算量越低，则矩阵链 $A_1 \dots A_n$ 所耗费的计算量越低
- 总共的计算量包括 $\text{cost}(A_1 \dots A_k) + \text{cost}(A_{k+1} \dots A_n) + \text{cost}$ （两个矩阵子链的乘积生成的矩阵相乘）。而无论子问题的解决方案如何，最后一项的计算量不变。



动态规划方法

- 将矩阵连乘积 $A(i)A(i+1)\dots A(j)$ 简记为 $A[i:j]$ ，设计算 $A[i:j]$ ($1 \leq i \leq j \leq n$)所需要的最少数乘次数 $m[i,j]$ ，则原问题的最优值为 $m[1,n]$
- 当 $i = j$ 时， $A[i:j] = A_i$ ，因此， $m[i,i] = 0$ ， $i = 1, 2, \dots, n$
- 当 $i < j$ 时， $m[i,j] = m[i,k] + m[k+1,j] + p(i-1)p(k)p(j)$
- 这里 $A(i)$ 的维数为 $p(i-1) \times p(i)$ (注： $p(i-1)$ 为矩阵 $A(i)$ 的行数， $p(j)$ 为矩阵 $A[j]$ 的列数)
- 可以递归地定义 $m[i,j]$ 为：

$$m[i,j] = \begin{cases} 0 & \text{if } i = j \\ \min_{i \leq k < j} \{m[i,k] + m[k+1,j] + p[i-1]p[k]p[j]\} & \text{if } i < j \end{cases}$$

2025-10-15

38
华中农业大学信息学院



例子

The length of chain is 5, and the dimensions is:

$A_1 5 \times 10, A_2 10 \times 4, A_3 4 \times 6, A_4 6 \times 10, A_5 10 \times 2$

$m[1,1]=0$	$m[1,2]=200$	$m[1,3]=320$	$m[1,4]=620$	$m[1,5]=348$
	$m[2,2]=0$	$m[2,3]=240$	$m[2,4]=640$	$m[2,5]=248$
		$m[3,3]=0$	$m[3,4]=240$	$m[3,5]=168$
			$m[4,4]=0$	$m[4,5]=120$
				$m[5,5]=0$

华中农业大学信息学院



算法

- 下面的伪代码假定矩阵 A_i 尺寸 $p(i-1) \times p(i)$ for $i = 1, 2, \dots, n$.
- 输入是 $p = p[0], p[1], \dots, p[n]$, 其中 $length[p] = n + 1$.
- 该过程使用的辅助表 $m[1..n, 1..n]$ 用于存储的最小 $m[i, j]$ 的计算耗费
- 辅助表 $s[1..n, 1..n]$ 记录索引 k 达到最佳的成本计算时的 $m[i, j]$, 我们将使用表 S 构建一个最优方案。



Algorithm MATRIX-CHAIN-ORDER(p)

$n \leftarrow \text{length}[p] - 1$ // n 个矩阵

for $i \leftarrow 1$ **to** n

$m[i, i] \leftarrow 0$

end for

for $l \leftarrow 2$ **to** n $\triangleright$ l is the chain length.

for $i \leftarrow 1$ **to** $n - (l - 1)$

$j \leftarrow i + (l - 1)$

$m[i, j] \leftarrow \infty$

for $k \leftarrow i$ **to** $j - 1$

$q \leftarrow m[i, k] + m[k + 1, j] + p[i-1] p[k] p[j]$ // 矩阵乘法次数

if $q < m[i, j]$ // 最小数乘次数

then $m[i, j] \leftarrow q, s[i, j] \leftarrow k$

end if

end for

end for

end for

return m and s

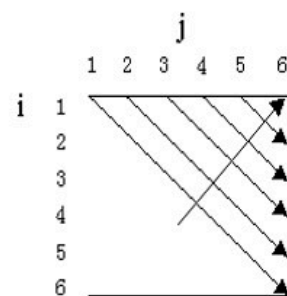
2025-10-15

41
华中农业大学信息学院



例子

A_1	A_2	A_3	A_4	A_5	A_6
30×35	35×15	15×5	5×10	10×20	20×25



(a) 计算次序

	j	1	2	3	4	5	6
i	1	0	15750	7875	9375	11875	15125
	2		0	2625	4375	7125	10500
	3			0	750	2500	5375
	4				0	1000	3500
	5					0	5000
	6						0

(b) $m[i][j]$

	j	1	2	3	4	5	6
i	1	0	1	1	3	3	3
	2		0	2	3	3	3
	3			0	3	3	3
	4				0	4	5
	5					0	5
	6						0

(c) $s[i][j]$

华中农业大学信息学院



算法分析

- The loops are nested three deep, and each loop index (l , i , and k) takes on at most $n - 1$ values.
- 算法matrixChain的主要计算量取决于算法中对 r , i 和 k 的3重循环。循环体内的计算量为 $O(1)$, 而3重循环的总次数为 $O(n^3)$ 。
- 算法的时间复杂性是 $O(n^3)$. 算法的空间复杂度是 $\Theta(n^2)$ 用于存储 m 和 s 表.



7.5 所有点对的最短路径问题

? 问题描述:

- 设 $G=(V, E)$ 是一个有向图，其中的每条边 (i, j) 有一个非负的长度 $l[i, j]$ ，如果从顶点 i 到顶点 j 没有边，则 $l[i, j]=\infty$ 。
- 问题是要找出从每个顶点到其他所有顶点的距离，这里，从顶点 x 到顶点 y 的距离是指从 x 到 y 的最短路径的长度。



7.5 问题分析

- 如果有一个矩阵 $D=[d(ij)]$ ，其中 $d(ij)>0$ 表示 i 城市到 j 城市的距离。若 i 与 j 之间无路可通，那么 $d(ij)$ 就是无穷大。又有 $d(ii)=0$ 。编写一个程序，通过这个距离矩阵 D ，把任意两个城市之间的最短与其行径的路径找出来。
- 我们可以将问题分解，如何找出最短路径呢，这里还是用到动态规划的知识，对于任何一个城市而言， i 到 j 的最短距离不外乎存在经过 i 与 j 之间的 k 和~~不经过 k~~ 两种可能，所以可以令 $k=1, 2, 3, \dots, n$ (n 是城市的数目)，再检查 $d(ij)$ 与 $d(ik)+d(kj)$ 的值；在此 $d(ik)$ 与 $d(kj)$ 分别是目前为止所知道的 i 到 k 与 k 到 j 的最短距离，因此 $d(ik)+d(kj)$ 就是 i 到 j 经过 k 的最短距离。每当一个 k 查完了， $d(ij)$ 就是目前的 i 到 j 的最短距离。重复这一过程，最后当递归查完所有的 k 时， $d(ij)$ 里面存放的就是 i 到 j 之间的最短距离了。

2025-10-15

45
华中农业大学信息学院



递推式

- 设 i 和 j 是 V 中两个不同的顶点,定义 $d_{i,j}^k$ 是从 i 到 j , 并且不经过 $\{k+1, k+2, \dots, n\}$ 中任何顶点的最短路径的长度。
- $d_{i,j}^0 = l[i, j]$, $d_{i,j}^1$ 是从 i 到 j , 除了可能经过顶点1以外, 不经过任何其他顶点的最短路径。 $d_{i,j}^2$ 是从 i 到 j , 除了可能经过顶点1、顶点2或同时经过它们以外, $d_{i,j}^n$ 不经过任何其他顶点的最短路径, 等等。这样由定义可知, $d_{i,j}^n$ 是从 i 到 j 的最短路径长度, 也就是从 i 到 j 的距离。给出这个定义, 可以递归地计算 $d_{i,j}^n$

$$d_{i,j}^k = \begin{cases} l[i, j] & \text{if } k = 0 \\ \min \{d_{i,j}^{k-1}, d_{i,k}^{k-1} + d_{k,j}^{k-1}\} & \text{if } 1 \leq k \leq n \end{cases}$$



基本思想

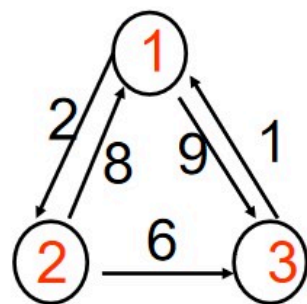
- 下面是Floyd设计出的算法，用自底向上地解上面的递推式的方法来处理。它用 $n+1$ 个 $n \times n$ 维矩阵 $D_0, D_1, \dots, D_n$ 来计算最短约束路径的长度。
- 开始时，如果 $i \neq j$ 并且 (i,j) 是 G 中的边，则置 $D_0[i,i] = 0$ ， $D_0[i,j] = l[i,j]$ ；否则置 $D_0[i,j] = \infty$ 。然后做 n 次迭代，使在第 k 次迭代后， $D_k[i,j]$ 含有从顶点 i 到顶点 j ，且经过编号大于 k 的任何顶点的最短路径的长度。这样在第 k 次迭代中，可以用公式计算 $D_k[i,j]$ ：

$$D_k[i,j] = \min\{D_{k-1}[i,j], D_{k-1}[i,k] + D_{k-1}[k,j]\}$$



例子

考虑图7.4所示的有向图



矩阵 D_0, D_1, D_2
and D_3 是:

$$D_0 = \begin{bmatrix} 0 & 2 & 9 \\ 8 & 0 & 6 \\ 1 & \infty & 0 \end{bmatrix}$$

$$D_1 = \begin{bmatrix} 0 & 2 & 9 \\ 8 & 0 & 6 \\ 1 & 3 & 0 \end{bmatrix}$$

$$D_2 = \begin{bmatrix} 0 & 2 & 8 \\ 8 & 0 & 6 \\ 1 & 3 & 0 \end{bmatrix}$$

$$D_3 = \begin{bmatrix} 0 & 2 & 8 \\ 7 & 0 & 6 \\ 1 & 3 & 0 \end{bmatrix}$$

2025-10-15

48
华中农业大学信息学院



Character of the recursive equation

- 一个重要的发现是，在第 k 次迭代中第 k 行和第 k 列都是不变的，因此可以仅用 D 矩阵的一个副本来进行计算。仅用一个 $n \times n$ 矩阵执行这个计算的算法由算法FLOYD给出。



算法7.3 FLOYD

输入: $n \times n$ 维矩阵 $l[1..n, 1..n]$, 对于有向图 $G=(\{1, 2, \dots, n\}, E)$ 中的边 (i, j) 的长度是 $l[i, j]$ 。

输出: 矩阵 D 使得 $D[i, j]$ = i 到 j 的距离。

```
1.  $D \leftarrow l$       {将输入矩阵  $l$  复制到  $D$ }
2. for  $k \leftarrow 1$  to  $n$ 
3.   for  $i \leftarrow 1$  to  $n$ 
4.     for  $j \leftarrow 1$  to  $n$ 
5.        $D[i, j] = \min\{D[i, j], D[i, k] + D[k, j]\}$ 
6.     end for
7.   end for
8. end for
```

- 显然, 算法的运行时间是 $\Theta(n^3)$, 它的空间复杂性是 $\Theta(n^2)$ 。



Floyd与Dijkstra算法比较

- Dijkstra算法,图所有边权值都为非负的; Floyd算法,不允许所有权值为负的回路,
- Dijkstra只可以求出任意点到达源点的最短距离; Floyd可以求出任意两点间的最短距离,
- Dijkstra算法的思想是贪心, Floyd算法的思想是动态规划
- Dijkstra时间复杂度 $O(n^2)$, Floyd算法 $O(n^3)$
- Dijkstra 算法 在网络中用得更多, 一个一个节点添加, 加一个点刷一次路由表。Floyd 算法把所有已经连接的路径都标出来, 再通过不等式比较来更改路径。



动态规划基本步骤

- 找出最优解的性质，并刻画其结构特征。
- 递归地定义最优值。
- 以自底向上的方式计算出最优值。
- 根据计算最优值时得到的信息，构造最优解。

2025-10-15

52
华中农业大学信息学院



动态规划算法的基本要素1(最优子结构)

- 矩阵连乘计算次序问题的最优解包含着其子问题的最优解。这种性质称为**最优子结构性质**。
- 在分析问题的最优子结构性质时，所用的方法具有普遍性：首先假设由问题的最优解导出的子问题的解不是最优的，然后再设法说明在这个假设下可构造出比原问题最优解更好的解，从而导致矛盾。
- 利用问题的最优子结构性质，以自底向上的方式递归地从子问题的最优解逐步构造出整个问题的最优解。
最优子结构是问题能用动态规划算法求解的前提。

同一个问题可以有多种方式刻画它的最优子结构，有些表示方法的求解速度更快（空间占用小，问题的维度低）



动态规划算法的基本要素2(重叠子问题)

- 递归算法求解问题时，每次产生的子问题并不总是新问题，有些子问题被反复计算多次。这种性质称为**子问题的重叠性质**。
- 动态规划算法，对每一个子问题只解一次，而后将其解保存在一个表格中，当再次需要解此子问题时，只是简单地用常数时间查看一下结果。
- 通常不同的子问题个数随问题的大小呈多项式增长。因此用动态规划算法只需要多项式时间，从而获得较高的解题效率。

2025-10-15

54
华中农业大学信息学院



Exercises

- 7.5
- 7.9
- 7.15
- 7.21

2025-10-15

55
华中农业大学信息学院